

August 2026

Asymmetry Score

D_{asym} is the Randers-perturbed geodesic distance matrix (asymmetric: $D_{\text{asym}}[i, j] \neq D_{\text{asym}}[j, i]$ in general), and $\text{bln}[i, j]$ marks the real, directly constructed graph edges. The score is computed only over these direct neighbours, not all $n - 1$ pairs. For node i and each direct neighbour j with $\text{bln}[i, j] = \text{True}$:

$$\text{ratio}_{ij} = \frac{|D_{\text{asym}}[i, j] - D_{\text{asym}}[j, i]|}{D_{\text{asym}}[i, j] + D_{\text{asym}}[j, i]} \in [0, 1)$$

$\text{ratio}_{ij} = 0$ means the edge is perfectly symmetric ($D_{\text{asym}}[i, j] = D_{\text{asym}}[j, i]$); it approaches 1 only when one direction's distance collapses to 0 relative to the other.

$$\text{asym}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \text{ratio}_{ij}, \quad \mathcal{N}(i) = \{j : \text{bln}[i, j] = \text{True}, j \neq i\}$$

$$\text{asym}_{\text{global}} = \frac{1}{n} \sum_{i=1}^n \text{asym}_i$$

asym_i is a *mean* over i 's direct neighbours, not a sum – this keeps the score in $[0, 1)$ and comparable across different values of k . Table 1 reports $\text{asym}_{\text{global}}$ for the swiss-roll dataset across both adjacency rules and several values of k .

	k=10	k=20	k=30	k=40	k=50
ε threshold	0.3204	0.3206	0.3209	0.2985	0.2942
k-nearest-neighbours	0.3212	0.3211	0.3211	0.3203	0.3210

Table 1: asymmetry scores of swiss roll for different numbers of neighbourhood

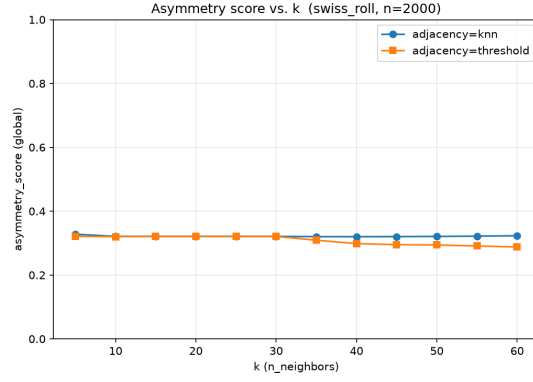


Figure 1: Graph of Asymmetry Score vs Number of Neighbours for Swiss Roll

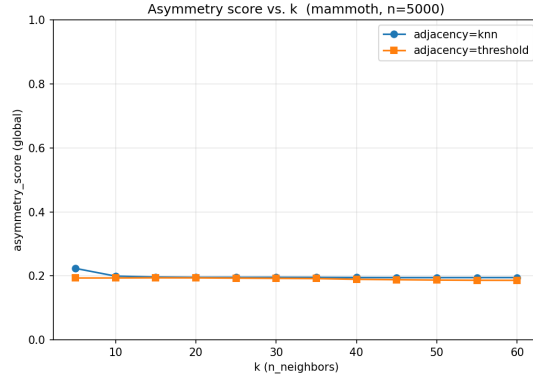


Figure 2: Graph of Asymmetry Score vs Number of Neighbours for Mammoth

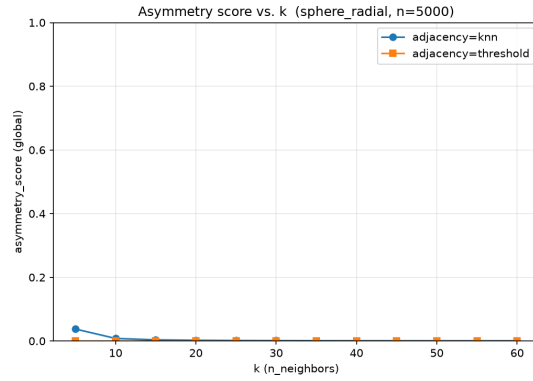


Figure 3: Graph of Asymmetry Score vs Number of Neighbours for Sphere with Radial Drifts

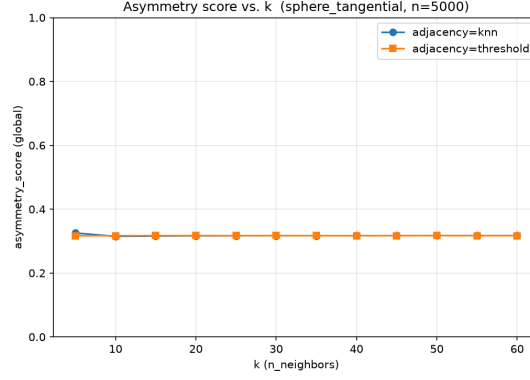


Figure 4: Graph of Asymmetry Score vs Number of Neighbours for Sphere with Tangential Drifts

Randers vs. Euclidean Force Toggles

In `randers_umap_fit`, the attraction, repulsion, and gravity terms can each independently use either the Randers-perturbed metric or the plain Euclidean one, via three booleans: `randers_attractive`, `randers_repulsive`, `randers_gravity`. When a term's flag is `True`, it uses $\rho_r = d + \langle B, y_j - y_i \rangle$ and $g_r = e + B$ (the drift B perturbs both the effective distance and direction); when `False`, it falls back to the plain Euclidean $\rho_e = d$, $g_e = e$, ignoring B entirely for that term. Gravity works the same way, but pulls node i toward its own virtual target $y_i + b_i$ rather than toward another node. The figures below sweep gravity strength under different combinations of these three flags.

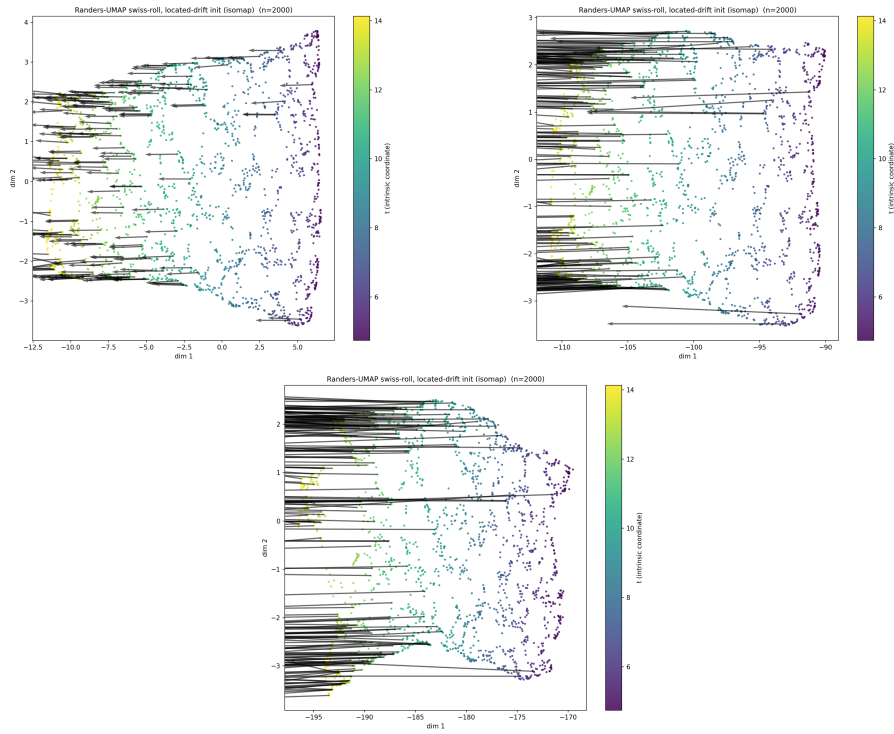


Figure 5: swiss roll embedding with gravity strength 0, 5, 10 with `grav_randers=True`, `attr_randers=False`, `rep_randers=False`

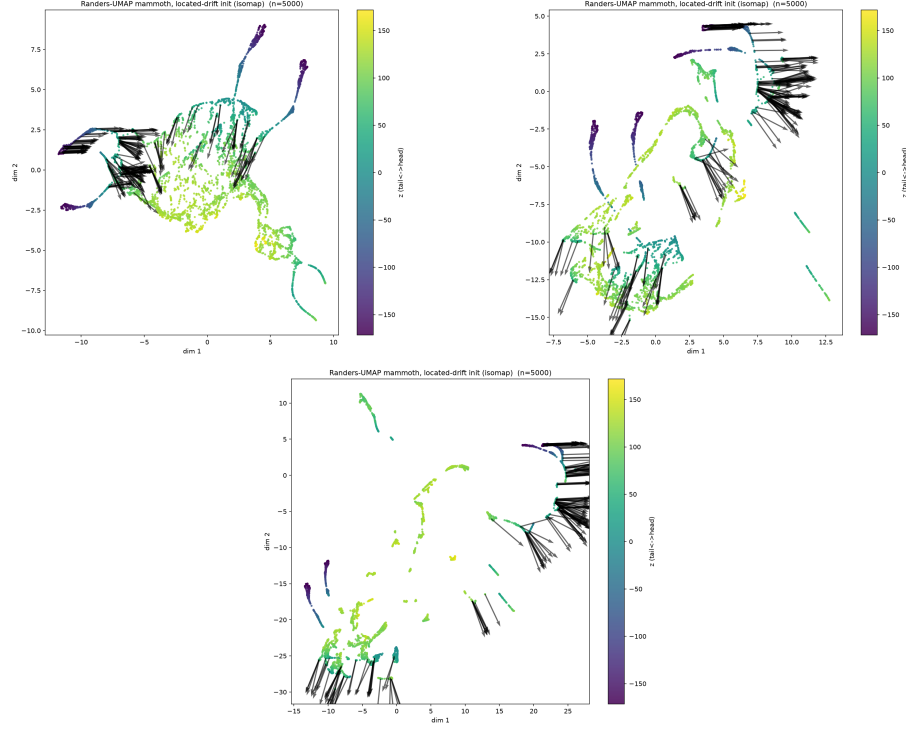


Figure 6: mammoth embedding with gravity strength 0, 5, 10 with `randers_gravity=True`, `randers_attractive=False`, `randers_repulsive=False`

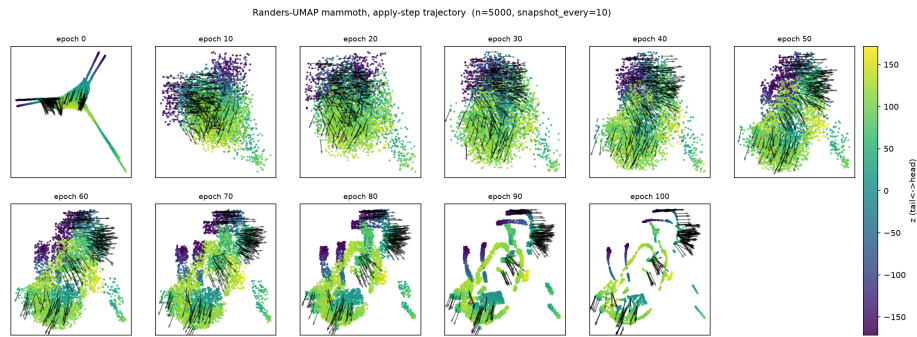


Figure 7: Snapshots of mammoth embedding for strength 5 gravity

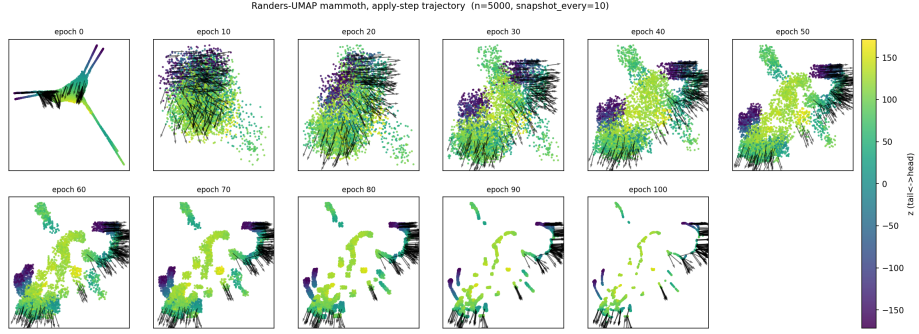


Figure 8: Snapshots of mammoth embedding for strength 10 gravity

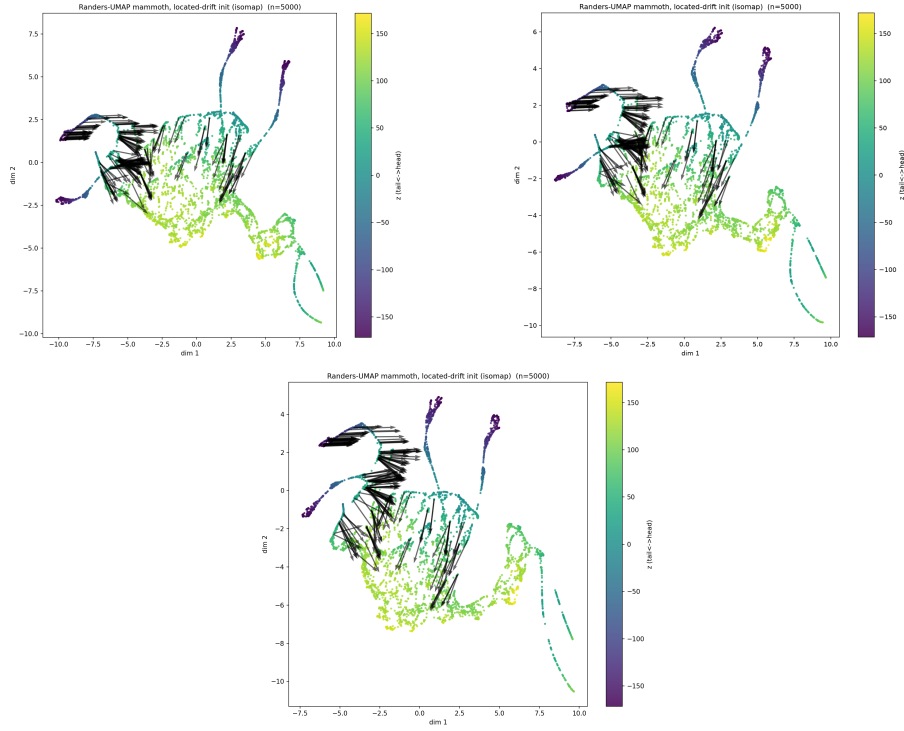


Figure 9: mammoth embedding with gravity strength 0, 5, 10 with `randers_gravity=False`, `randers_attractive=True`, `randers_repulsive=False`

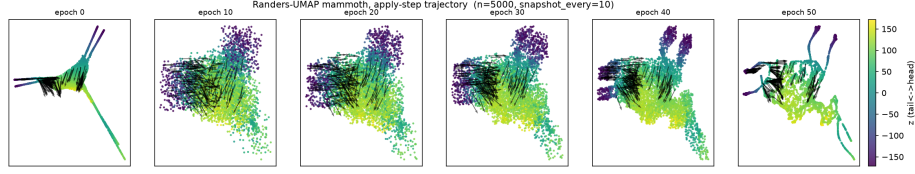


Figure 10: Snapshots of mammoth embedding for strength 5 gravity

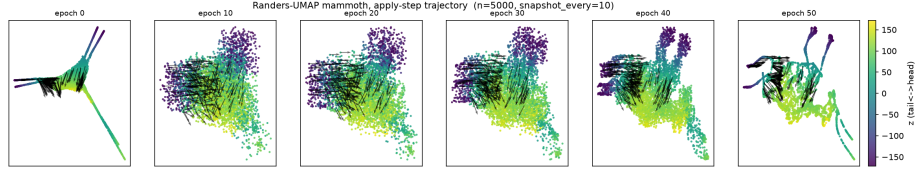


Figure 11: Snapshots of mammoth embedding for strength 10 gravity

Drift Magnitude Normalization

The drift vector B_i that feeds the Randers terms above is frozen once, at the "locate" step, before training. This section compares how its magnitude is treated afterwards: a fixed clipped constant (before) versus a live, embedding-relative rescaling (now, via `--normalize`). **Before: clip_delta-based, frozen for the whole run**

$$B_i^{(0)} = \hat{Y}_i^{\text{virt}} - \hat{Y}_i^{\text{real}}$$

Locate step: a single deterministic placement, computed once, before any training.

$$B_i = B_i^{(0)} \cdot \min\left(1, \frac{1 - \delta}{\|B_i^{(0)}\|}\right), \quad \delta = \text{clip_delta (default 0.01)}$$

$$\implies \|B_i\| \leq 1 - \delta \approx 1$$

A fixed, embedding-scale-independent constant.

$$B_i(t) = B_i \quad \forall t = 1, \dots, T$$

Both direction *and* magnitude are constant for the whole training run. **Now:**

`scale_B_fixed_by_knn_distance (--normalize)`

$$\hat{b}_i = \frac{B_i^{(0)}}{\|B_i^{(0)}\|}$$

Direction is still frozen, exactly as before – unaffected by everything below.

$$d_i^{(k)}(t) = \max_{j \in \text{kNN}_k(i, Y(t))} \|y_i(t) - y_j(t)\|$$

Distance to node i 's k -th nearest neighbour in the *live* embedding $Y(t)$.

$$B_i(t) = \frac{\|B_i^{(0)}\|}{d_i^{(k)}(t)} \cdot \hat{b}_i$$

Recomputed every epoch t . The magnitude is the *frozen located magnitude* divided by the live k -th-nearest-neighbour distance.

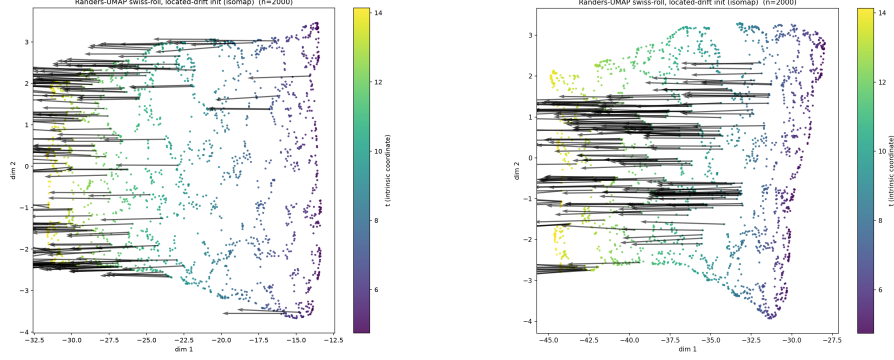


Figure 12: results of swiss roll embedding without normalization and with normalization (epochs 100)

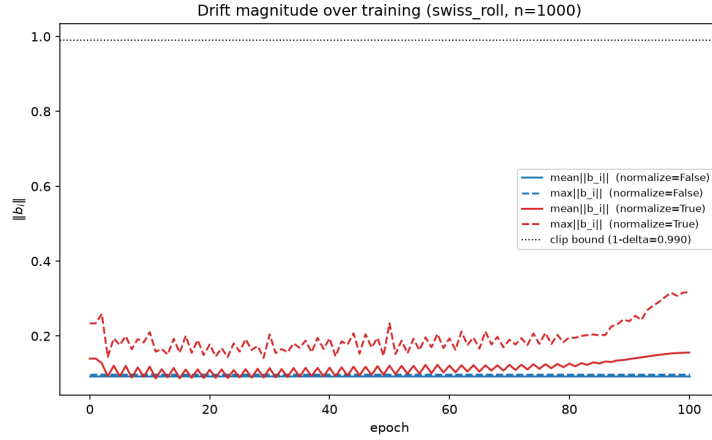


Figure 13: Drift Magnitude Graph for Swiss Roll Embedding

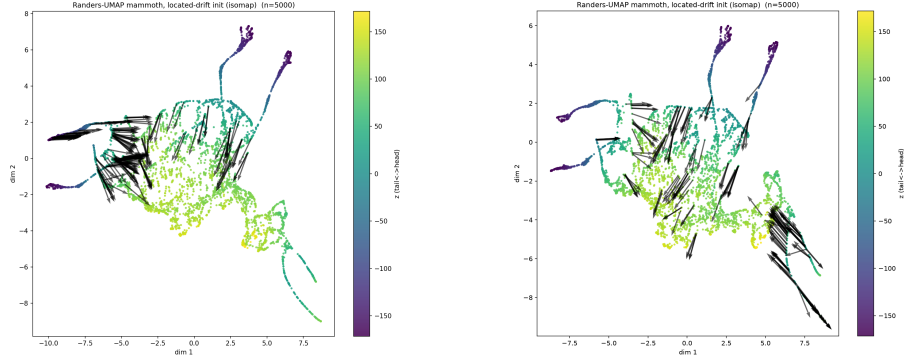


Figure 14: results of mammoth embedding without normalization and with normalization (epochs 50)

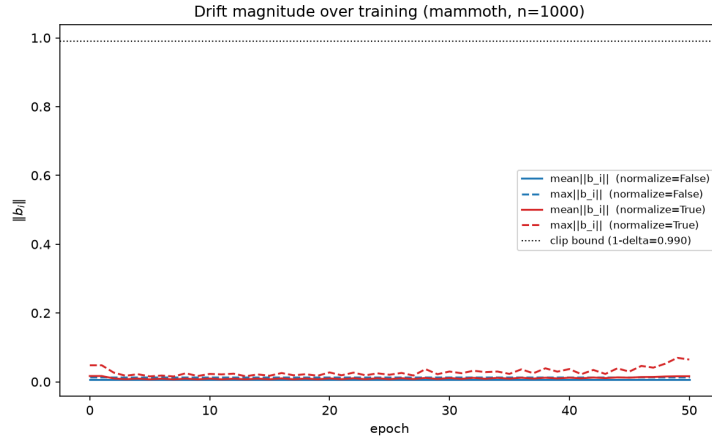


Figure 15: Drift Magnitude Graph for Mammoth Embedding

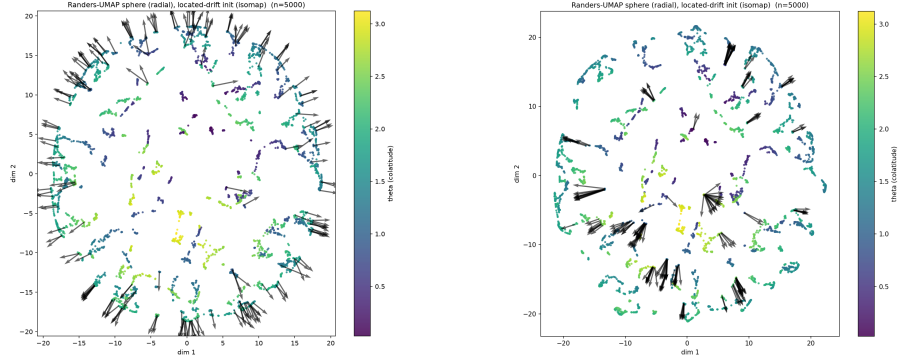


Figure 16: results of sphere with radial drifts embedding without normalization and with normalization (epochs 50)

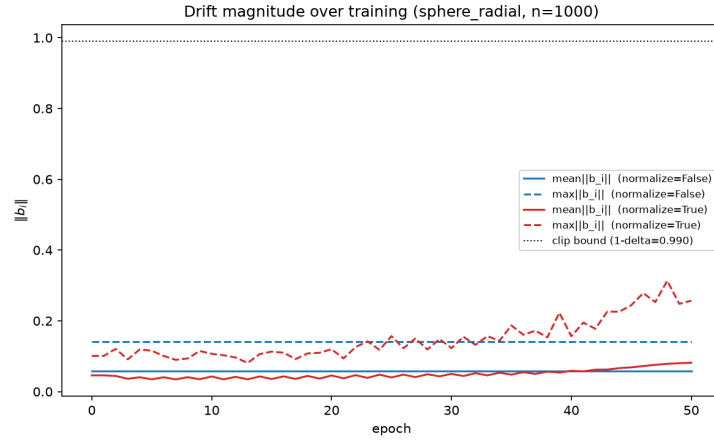


Figure 17: Drift Magnitude Graph for Sphere with Radial Drifts Embedding

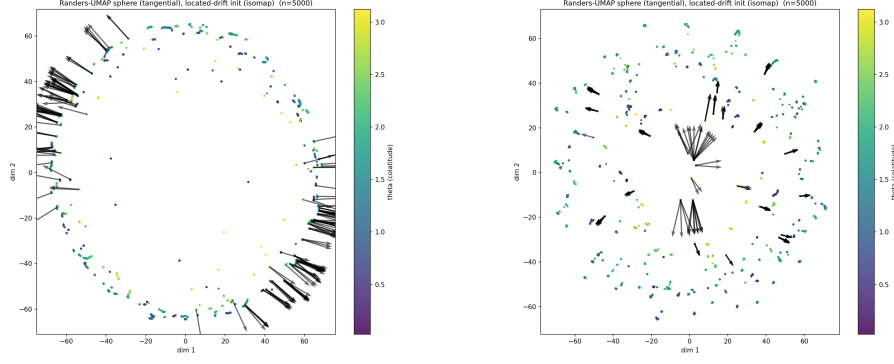


Figure 18: results of sphere with tangential drifts embedding without normalization and with normalization (epochs 50)

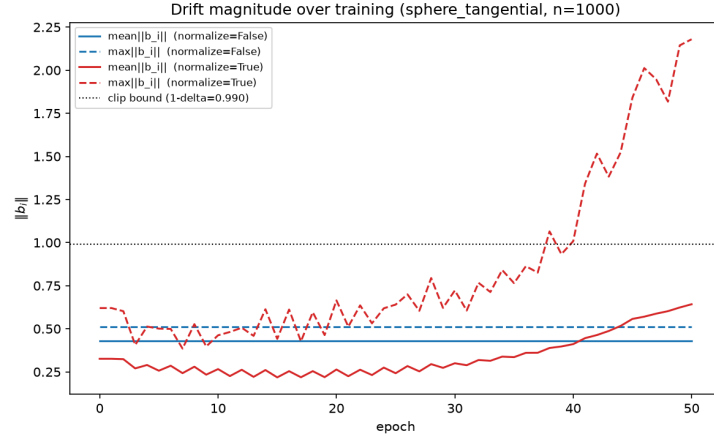


Figure 19: Drift Magnitude Graph for Sphere with Tangential Drifts Embedding

MNIST: Raw vs. PCA-First Pipelines

Motivation. MNIST offers a natural annotated ($n = 70,000$, 784-dimensional, 10-class) test bed beyond the synthetic manifolds used elsewhere in this report. Two ways of applying our method were suggested: (1) run the pipeline directly on the raw 784-dimensional pixel space, or (2) linearly project to a lower dimension first (van der Maaten’s classic t -SNE-preprocessing recommendation, ~ 50 dimensions) and run the pipeline on the projection. Both were implemented as a controlled pair, `embed_MNIST_raw.py` and `embed_MNIST_pca.py`, sharing every stage of the pipeline except the presence of the PCA step, so that any difference in the resulting embeddings can be attributed to the ambient dimensionality alone.

Shared pipeline. Let $X \in \mathbb{R}^{n \times 784}$ (raw) or $X_{\text{pca}} \in \mathbb{R}^{n \times d}$, $d = 50$ (PCA-reduced), denote the input to the following identical stages.

1. **Local asymmetric neighbourhood distance.** `IsUMap`’s own `distance_graph_generation` routine (unmodified, vendored library code) is run with $k = 30$ neighbours and per-point normalisation. It returns a dictionary R keyed by triples (i, j, k) , meaning “distance from j to k as measured within i ’s neighbourhood”; the genuine, directly measured distance from a point i to one of its own neighbours is stored exactly when $j = i$, with the third index giving the true target.
2. **Matrix construction.** A sparse asymmetric matrix is built by keeping only the $i = j$ entries of R and writing them as $D_{\text{sparse}}[i, k] = R[(i, i, k)]$, with every unmeasured pair left as $+\infty$. (An earlier version of this step wrote every (i, j, k) triple to position $[i, j]$ regardless of the $i = j$ condition; this landed real distances on the diagonal and unrelated triangulated values at the wrong indices, leaving $\sim 95\%$ of the matrix at a spurious literal zero. Since nearest-neighbour search always prefers the smallest value, the resulting neighbour graph was pure noise — verified to have 0% overlap with the true nearest neighbours on a controlled synthetic test, and to be the direct cause of an embedding with no visible digit clusters regardless of epoch count. The fix above restores 100% overlap on the same test.)
3. **Directed completion.** D_{sparse} , still sparse ($\sim k$ finite entries per row), is completed into a dense, connected, still-asymmetric matrix $D_{\text{asym}} \in \mathbb{R}^{n \times n}$ via directed Dijkstra (`scipy.sparse.csgraph.shortest_path, directed=True`) — the same completion strategy used by `randers_bridge.compute_dist_matrix` elsewhere in this project.
4. **Embedding.** D_{asym} is passed directly to `randers_umap_fit` ($n_{\text{neighbors}} = 20$, 10 negative samples, 500 epochs, `use_drift=True`, $B_{\text{fixed}} = \text{None}$). The drift field B is therefore not fixed in advance but recovered live every epoch from D_{asym} ’s own asymmetry via `compute_drift`, with `norm_mode="relative"` guaranteeing $\|b_i\| < 1$ by construction — no virtual points and no class-centroid targets are involved.

Where the two variants differ. `embed_MNIST_pca.py` inserts a single step before stage 1: $X \mapsto X_{\text{pca}} = \text{PCA}_d(X)$, $d = 50$ by default, and stage 1 is then run on X_{pca} instead of X . `embed_MNIST_raw.py` omits this step entirely and runs stage 1 directly on the full 784-dimensional X . No other stage, hyperparameter, or force toggle differs between the two scripts.

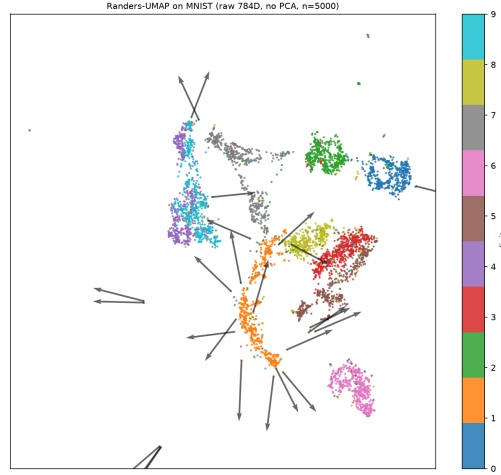


Figure 20: MNIST Embedding on 28x28-dim with 300 Epochs

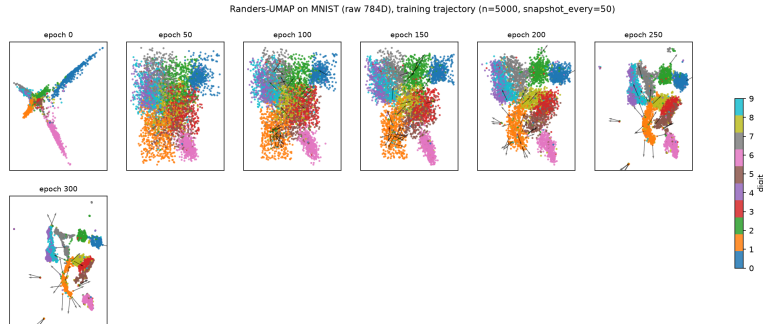


Figure 21: Snapshots of MNIST Embedding on 28x28-dim with 300 Epochs

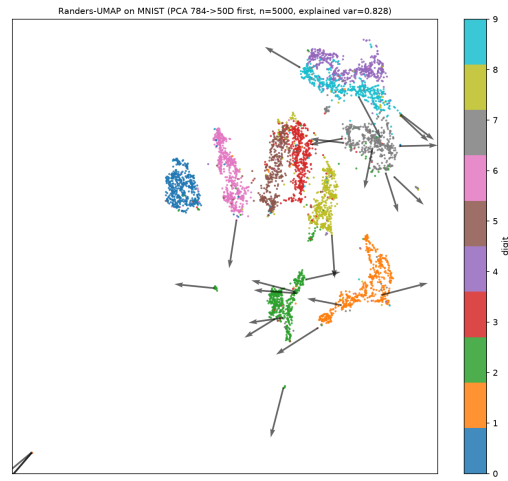


Figure 22: MNIST Embedding on 50-dim with 300 Epochs



Figure 23: Snapshots of MNIST Embedding on 50-dim with 300 Epochs

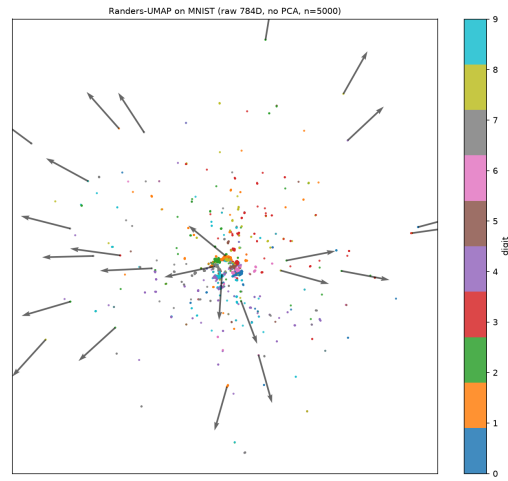


Figure 24: MNIST Embedding on 28x28-dim with 300 Epochs with gravity of strength 5

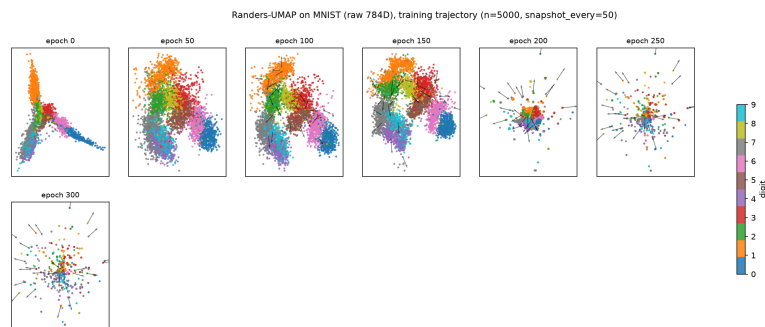


Figure 25: Snapshots of MNIST Embedding on 28x28-dim with 300 Epochs with gravity of strength 5

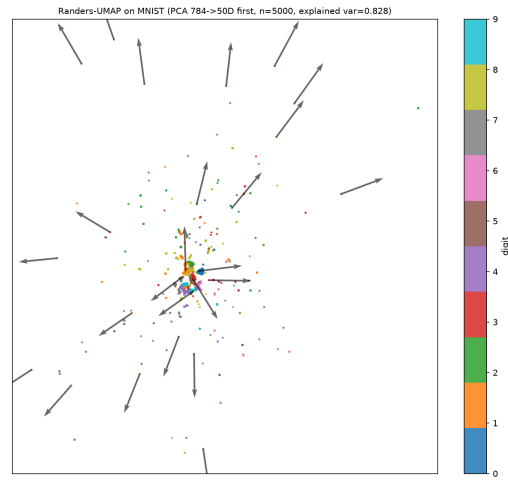


Figure 26: MNIST Embedding on 50-dim with 300 Epochs with gravity of strength 5

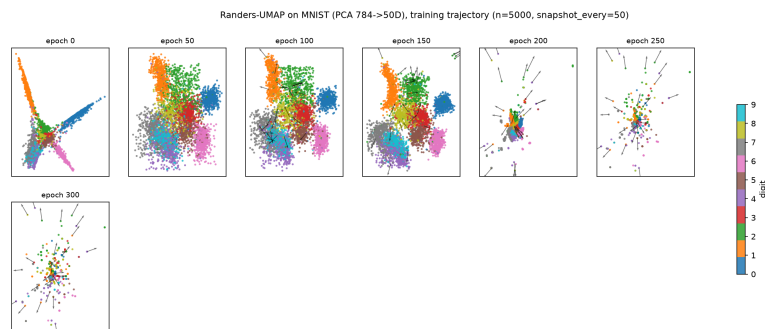


Figure 27: Snapshots of MNIST Embedding on 50-dim with 300 Epochs with gravity of strength 5

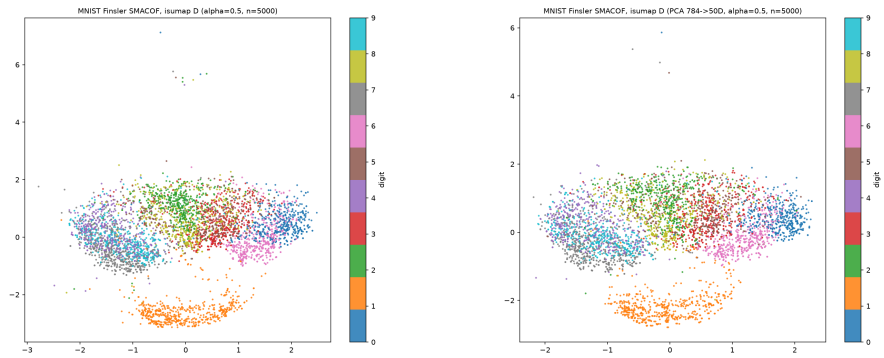


Figure 28: FinslerMDS results for MNIST Embedding without and with PCA

Wisconsin Breast Cancer Dataset

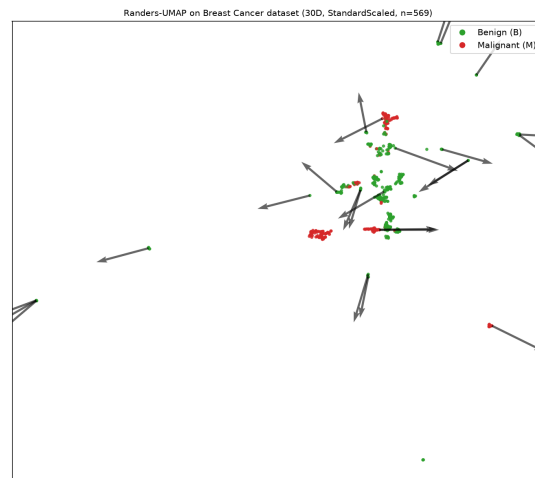


Figure 29: Breast Cancer Dataset Embedding with 100 Epochs

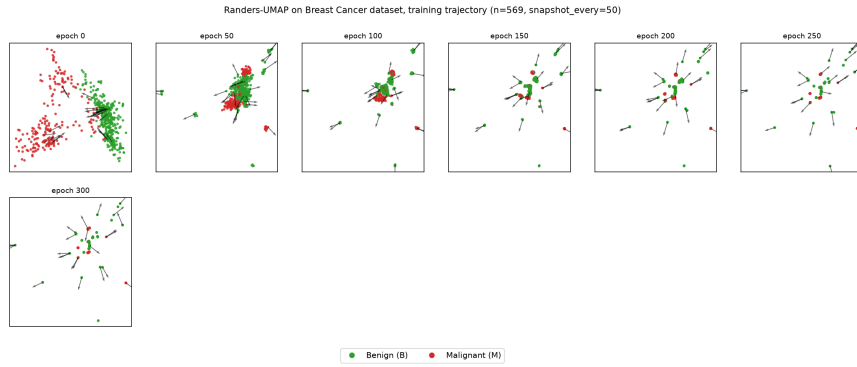


Figure 30: Breast Cancer Dataset Embedding Snapshots with 300 Epochs

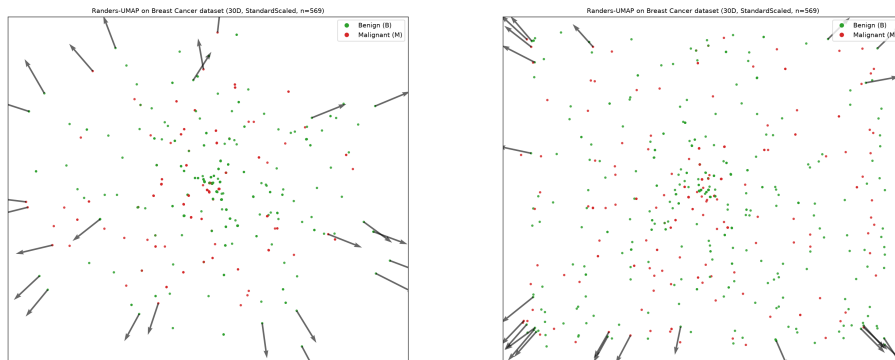


Figure 31: Breast Cancer Dataset Embedding with gravity of strength 5 and 10

scRNA Dataset

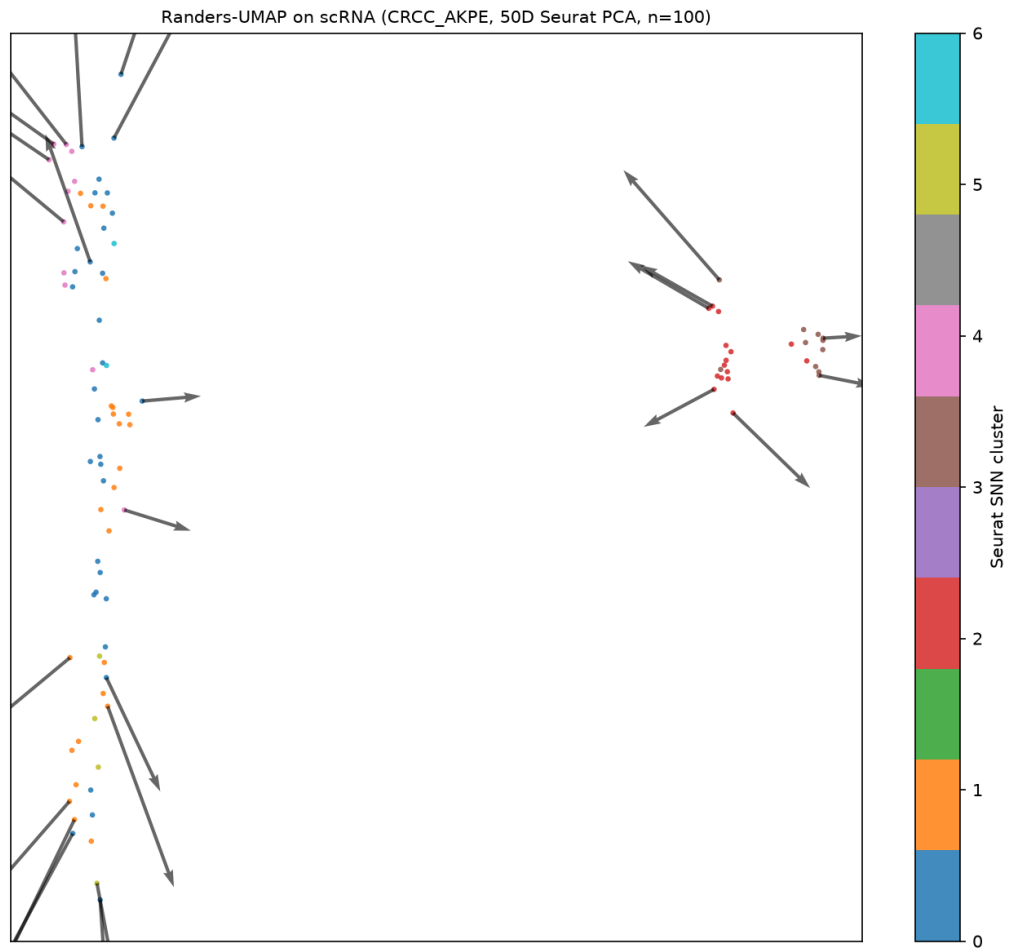


Figure 32: scRNA Dataset Embedding with 100 Epochs

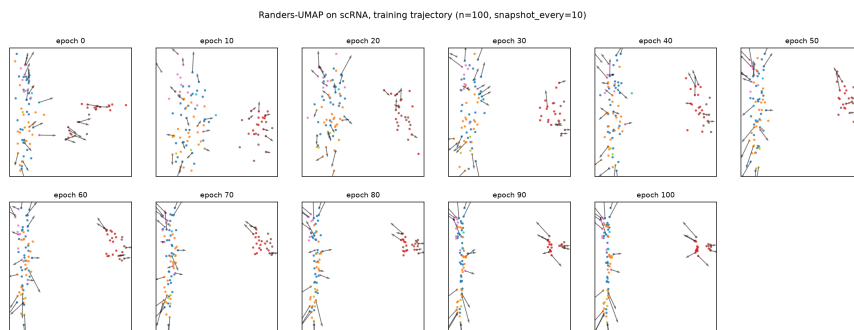


Figure 33: Snapshots of scRNA Dataset Embedding with 100 Epochs

Force Model: Fruchterman–Reingold Baseline (replacing the UMAP (a, b) -curve as default)

Until this change, the attraction/repulsion law driving the force-directed layout was UMAP’s own fitted (a, b) -curve (Section 3.2 of the UMAP paper), evaluated at the (possibly Randers-substituted) pair (ρ, g) :

$$\text{attr}(\rho) = \frac{2ab\rho^{2b-1}}{1+a\rho^{2b}}, \quad \text{rep}(\rho) = \frac{2b\rho}{\rho^2(1+a\rho^{2b})}, \quad (1)$$

weighted by the fuzzy membership $\mu \in [0, 1]$ (attraction) and $1 - \mu$ (repulsion), with a negative-sampling correction since every non-neighbour pair is summed densely rather than stochastically sampled.

Since the project’s gravity mechanism was itself ported from Bannister, Eppstein, Goodrich & Trott, “*Force-Directed Graph Drawing Using Social Gravity and Scaling*”, we implemented that paper’s own baseline attraction/repulsion law – the force the paper’s gravity term is *added to*, not UMAP’s – as an alternative, and made it the new default. Concretely, we added a `force_model` parameter to `randers_umap_fit` with two values:

- “`umap`” – the original (a, b) -curve law described above (unchanged, kept for comparison/ablation).
- “`fr_gravity`” (new default) – the paper’s own Fruchterman–Reingold-style law,

$$f_a(i, j) = \frac{\rho(i, j)}{k} g(i, j) \quad (\text{attraction, real edges only}), \quad (2)$$

$$f_r(i, j) = -\frac{k^2}{\rho(i, j)^2} g(i, j) \quad (\text{repulsion, all pairs}), \quad (3)$$

where k is the paper’s own natural-edge-length constant (default $k = 1/\sqrt{n}$, tunable via `fr_k`), and edges are the undirected union of the UMAP k -NN graph, $A = \text{knn_mask} \vee \text{knn_mask}^\top$, rather than a continuous membership weight, since the paper’s law is binary.

Everything else in the pipeline is unaffected by this choice: the drift vector B (live or frozen), the Randers substitution

$$\rho(i \rightarrow j) = \|y_i - y_j\| + b_i \cdot (y_j - y_i), \quad (4)$$

the gravity term itself, the virtual-neighbour mechanism, the ramp schedule, gradient clipping, and the decaying learning rate all apply identically under either force law – only the base attraction/repulsion *law* changes. The paper’s own temperature-cooling/threshold-triggered step schedule was deliberately *not* ported; we retain our existing optimisation scheme for both force models, isolating the comparison to the force law itself.

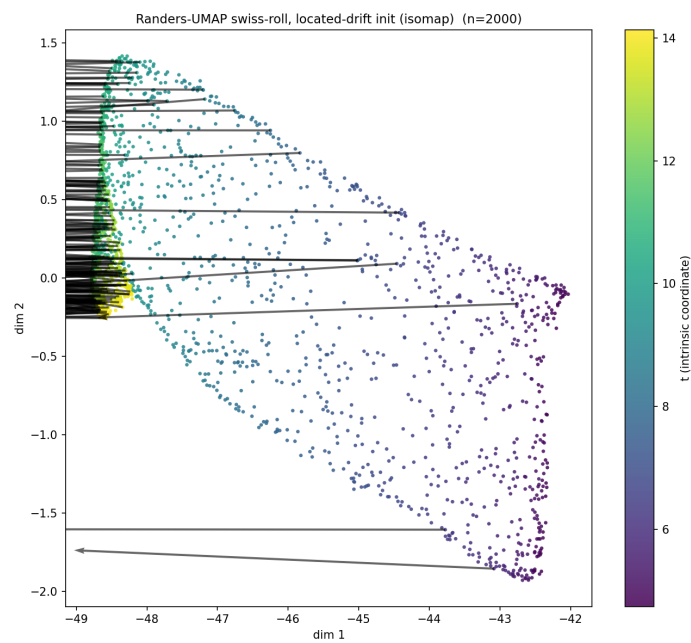


Figure 34: Swiss Roll Embedding with 200 epochs

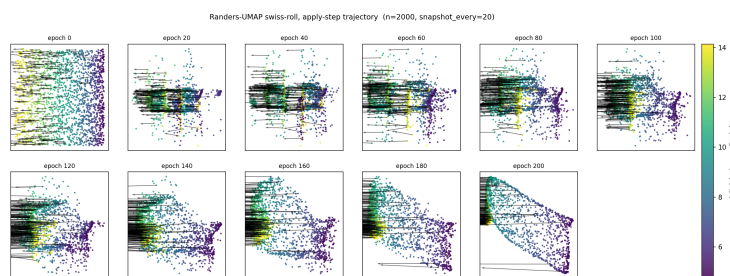


Figure 35: Snapshots of the Swiss Roll Embedding

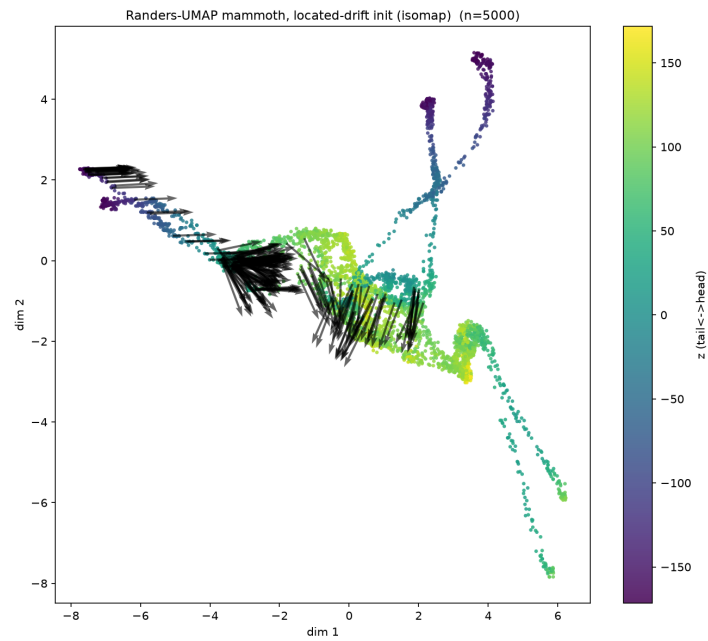


Figure 36: Mammoth Embedding with 50 epochs

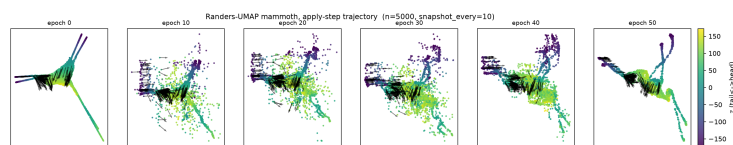


Figure 37: Snapshots of the Mammoth Embedding

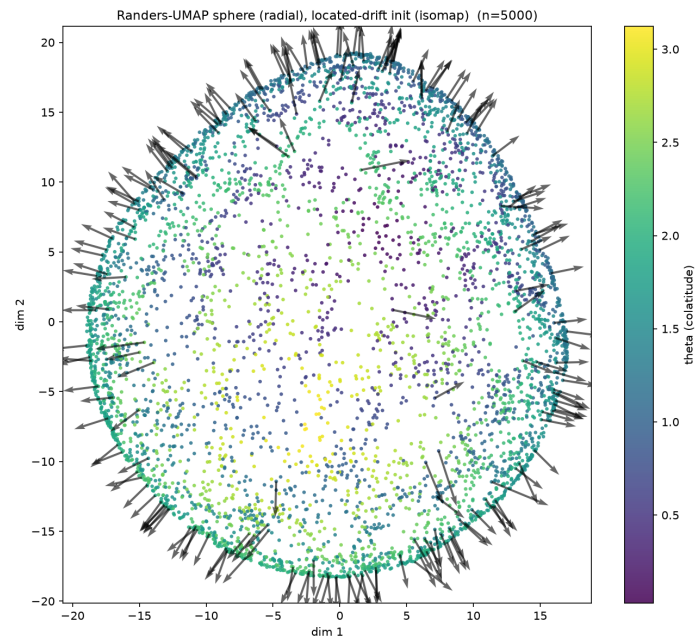


Figure 38: Sphere with Radial Drifts Embedding with 50 epochs

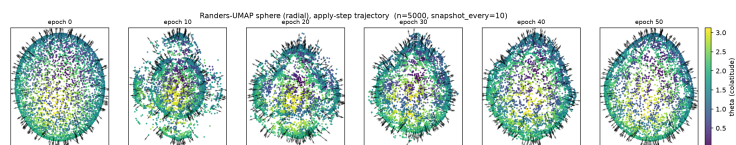


Figure 39: Snapshots of the Sphere with Radial Drifts Embedding

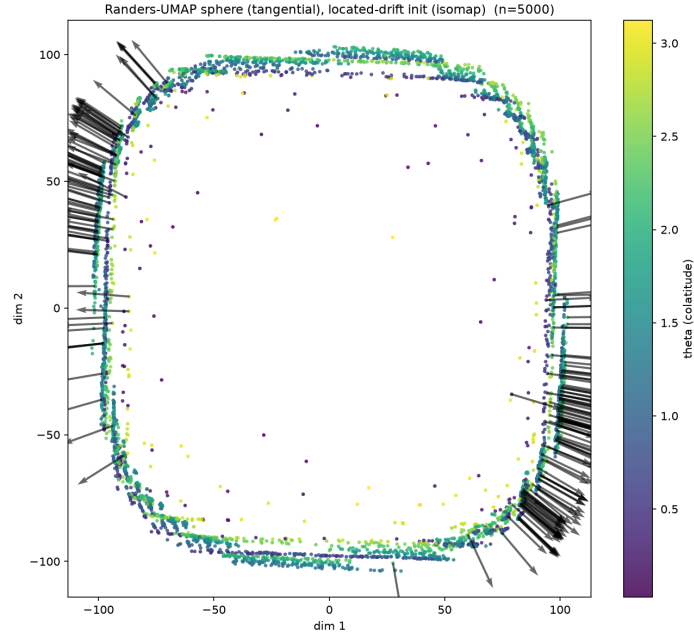


Figure 40: Sphere with Tangential Drifts Embedding with 50 epochs

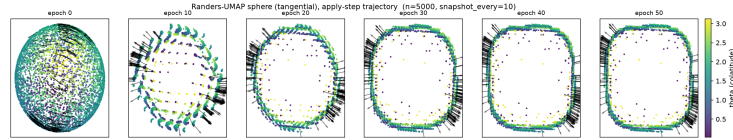


Figure 41: Snapshots of the Sphere with Tangential Drifts Embedding

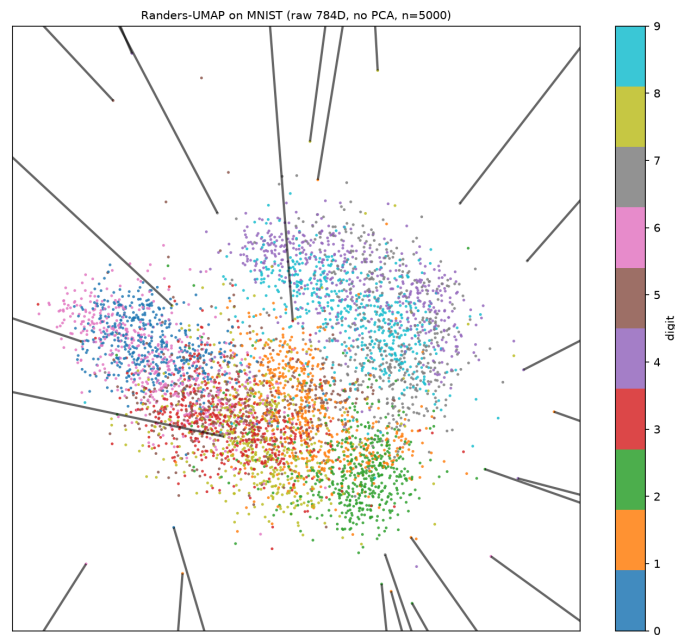


Figure 42: Raw MNIST Dataset Embedding with 300 epochs

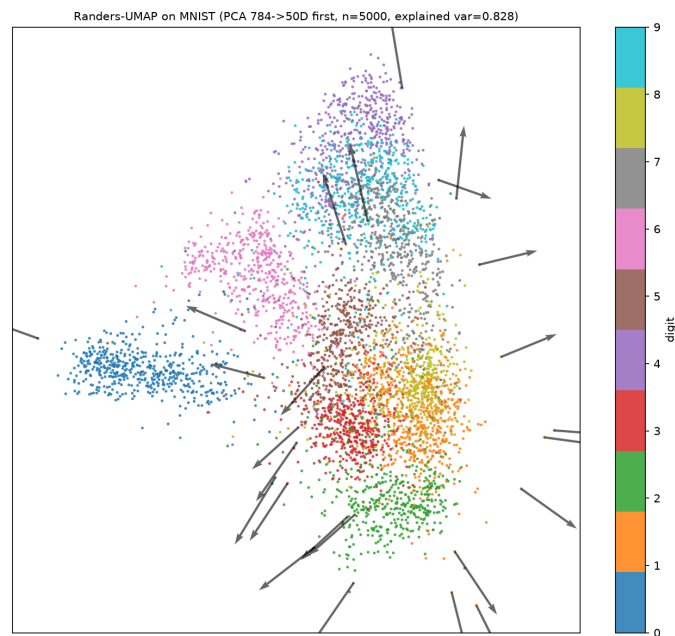


Figure 43: MNIST Dataset with pca-50 Embedding with 300 epochs

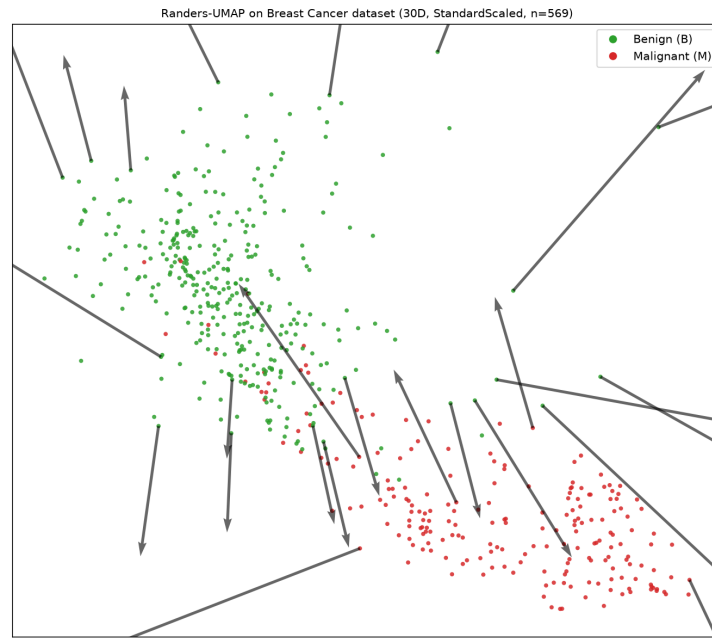


Figure 44: Breast Cancer Dataset Embedding with 100 epochs

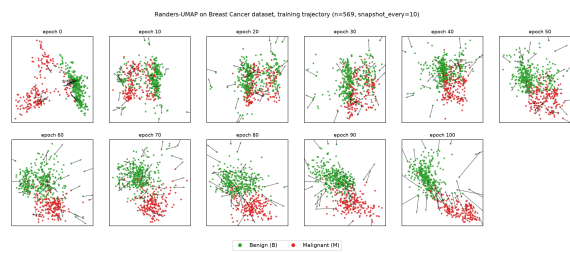


Figure 45: Snapshots of the Breast Cancer Dataset Embedding

Why `force_model="fr_gravity"` and `force_model="umap"` Produce Such Different Embeddings

The two force laws differ not just in a handful of coefficients but in their *asymptotic behaviour*, and this single difference explains nearly everything observed empirically below.

Asymptotics. UMAP’s attraction and repulsion, evaluated at (ρ, g) , both *decay* as $\rho \rightarrow \infty$:

$$\text{attr}(\rho) \sim \frac{2b}{\rho}, \quad \text{rep}(\rho) \sim \frac{2b}{a\rho^{1+2b}} \quad (\rho \rightarrow \infty), \quad (5)$$

so once two points have drifted far enough apart, essentially *no* force acts between them in either direction – nothing pulls them back together, and nothing pushes them further apart. Fruchterman–Reingold’s law behaves oppositely: repulsion decays only as the classical inverse square k^2/ρ^2 , while attraction *grows without bound*,

$$f_a(\rho) = \frac{\rho}{k} \quad (\rho \rightarrow \infty), \quad (6)$$

a Hookean spring whose restoring force increases linearly with distance. Consequently the FR-based layout always has a global force pulling stray points back toward the rest of the configuration, while UMAP’s layout has none: once a cluster or an individual point separates from the bulk, it is effectively free to drift.

Equilibrium spacing. At a real graph edge (i, j) , the FR force law’s net radial force is

$$f_a(\rho) - f_r(\rho) = \frac{\rho}{k} - \frac{k^2}{\rho^2}, \quad (7)$$

which vanishes exactly at $\rho = k$ – *every* edge is pulled toward the same equilibrium length k , independent of local density. UMAP’s law has no such shared equilibrium: attraction vanishes as $\rho \rightarrow 0$ and repulsion is already suppressed by $(1 - \mu) \approx 0$ at real edges (since $\mu \approx 1$ there), so each neighbourhood instead settles at whatever scale its own local density implies (via μ ’s ρ_i/σ_i calibration, `smooth_knn_dist`). This predicts that FR-based edge lengths should be far more *uniform* across the embedding than UMAP’s, and that UMAP should adapt better to genuinely non-uniform local density.

What each number measures. Before comparing the two force laws numerically, it helps to fix what each column in the tables below actually captures:

- **extent** – the spatial size of the final 2D layout (max minus min coordinate range). Larger extent means the embedding is more spread out, with points more likely to be flung far from the bulk.

- **edge_cv** – the coefficient of variation (std/mean) of the length of real graph edges in the embedding. Low = all edges end up roughly the same length (a uniform layout); high = some edges are tiny and others huge (an uneven, possibly fragmented layout).
- **asym_score** – **asymmetry_score** computed not on D_{asym} itself but on the *reconstructed* distances $\rho(i \rightarrow j) = \|y_i - y_j\| + b_i \cdot (y_j - y_i)$ from the trained embedding. It measures how much of the original target directional asymmetry survived training, under each force law.
- **end2end/edge** – (swiss roll only) the distance between the strip’s two true opposite ends, divided by the mean real-edge length. Checks whether the strip stays visibly ”unrolled” (large value) or curls back on itself (small value), in units comparable regardless of the layout’s overall scale.

Empirical confirmation (swiss roll, $n = 2000$, 200 epochs). Using `compare_force_models.py` on the same D_{asym} ($k = 20$), differing only in `force_model`:

	extent	edge_cv	asym_score	end2end/edge
fr_gravity	16.14	0.864	0.163	22.991
umap	74.63	2.528	0.167	3.915

Table 2: Force-model comparison on the swiss roll, $n = 2000$, 200 epochs (D_{asym} ’s own target `asymmetry_score` = 0.3206).

The extent gap (16.1 vs. 74.6, `umap` $\approx 4.6\times$ larger) is the clearest sign of the asymptotic argument above: with no restoring force at large ρ , `umap` keeps letting stray points drift further from the bulk, while `fr_gravity`’s unbounded Hookean attraction keeps pulling everything back toward a shared scale. **edge_cv** tells the same story from the edge-length side: FR’s real edges vary about a third as much in length (0.864 vs. 2.528), consistent with the shared $\rho = k$ equilibrium every edge is pulled toward, versus UMAP’s per-neighbourhood, density-dependent equilibrium.

asym_score is close between the two force laws here (0.163 vs. 0.167, i.e. about 51% vs. 52% of the 0.3206 target) – at this relatively early training stage (200 epochs), neither law has a clear edge in preserving the original directional signal.

end2end/edge is where `fr_gravity` looks most different from `umap` here (22.99 vs. 3.92): this reflects how far along training is, not a flaw in either model. `fr_gravity`’s global spring compacts the layout toward its $\rho = k$ equilibrium only gradually over many epochs, so at 200 epochs the strip is caught in a more elongated, less-compacted state – both a smaller **extent** in absolute terms and a comparatively much larger **end2end/edge**, since the mean edge length shrinks faster than the true end-to-end separation as training progresses. `umap`, having no comparable global equilibrium to converge toward, is far less sensitive to how many epochs the run is stopped at.

Summary. `force_model="fr_gravity"` yields a bounded, globally coherent, uniformly-spaced layout – well suited to a single continuous manifold such as the swiss roll, where global coherence matters more than fine local density adaptation. `force_model="umap"` yields better-separated local cluster structure on real, multi-modal data such as digits, at the cost of global coherence (the layout can fragment when nothing pulls stray points back). Which is preferable is therefore data-dependent, not a strict improvement in either direction. Note also that `fr_gravity`'s numbers (especially `extent` and `end2end/edge`) are noticeably more sensitive to how many training epochs are run than `umap`'s, since it takes time for its global spring to reach its $\rho = k$ equilibrium – this should be kept in mind when comparing runs at different epoch counts.

Negative Sampling for `fr_gravity`

Instead of summing repulsion over all $n - 1$ non-neighbours of a node, we draw n_{neg} random points per node, per epoch, and sum repulsion only over those:

$$j^{(1)}, \dots, j^{(n_{\text{neg}})} \sim \text{Uniform}(\{1, \dots, n\}), \quad (8)$$

redrawn independently every epoch. If a sample lands on the node itself ($j^{(s)} = i$), it is shifted to $(j^{(s)} + 1) \bmod n$ so no node ever repels itself.

For each sampled pair $(i, j^{(s)})$, the same distance/direction quantities used everywhere else in the pipeline are computed – including the Randers substitution when `randers_repulsive=True`, so drift still applies to sampled pairs exactly as it does to the dense sum:

$$\rho(i, j^{(s)}) = \|y_i - y_{j^{(s)}}\| + b_i \cdot (y_{j^{(s)}} - y_i). \quad (9)$$

The repulsion coefficient (`umap`'s $\text{rep}(\rho)$ or `fr_gravity`'s k^2/ρ^2) is evaluated at each sampled pair and summed into the node's total force update. For `fr_gravity`, this sampled sum is additionally rescaled by $(n - 1)/n_{\text{neg}}$ so that, in expectation, it equals the full dense sum over all $n - 1$ points – `umap`'s branch already carries an equivalent correction (`rep_scale`), so no separate rescale is needed there.

Everything else (attraction, gravity, drift, gradient clipping, the learning-rate schedule) is untouched – `negative_sampling` only changes how the repulsion sum is computed, not any other part of the update.

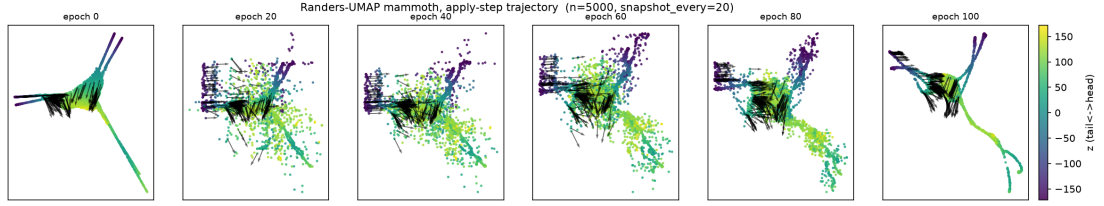


Figure 46: Snapshots of Mammoth Embedding with Negative Sampling

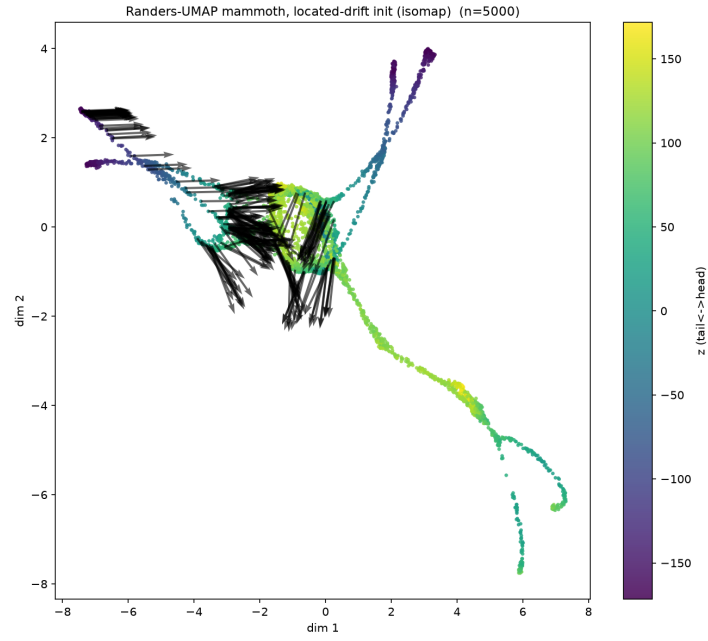


Figure 47: Mammoth Embedding with Negative Sampling

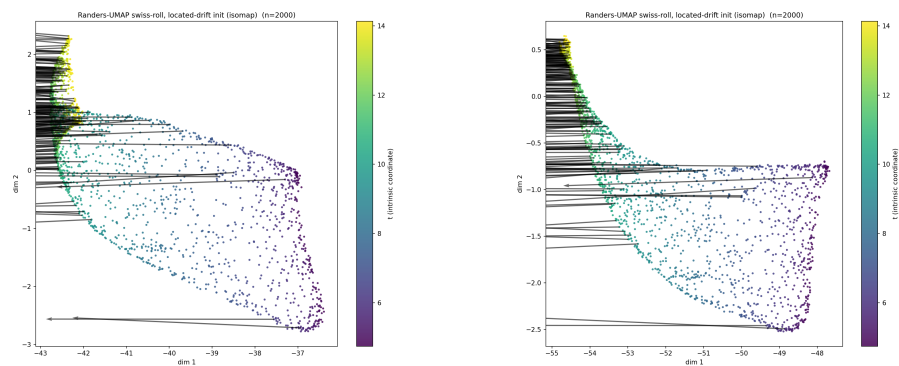


Figure 48: Swiss Roll Embedding with Negative Sampling for 200 and 300 epochs

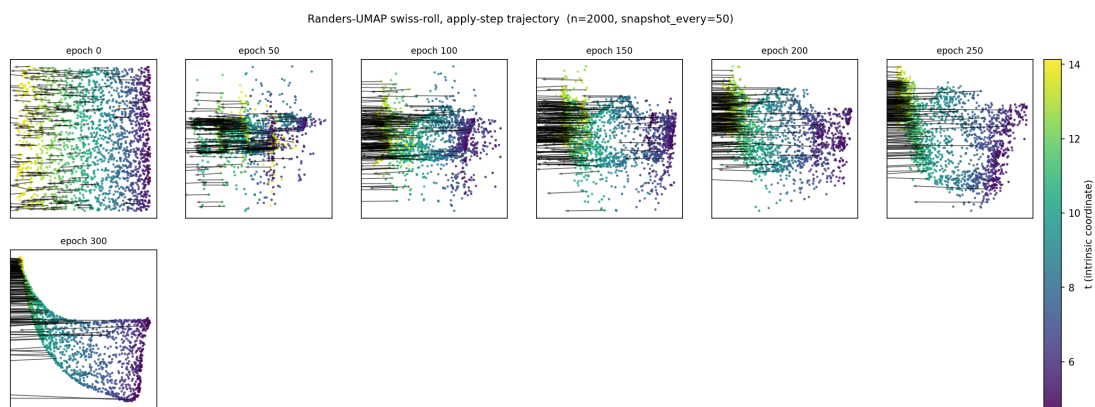


Figure 49: Snapshots of Swiss Roll Embedding with Negative Sampling